



Final Report

Introduction to Zero-Knowledge Proofs

Student Name: Nguyen Khanh Ly

Student ID: 20235600

Course: Cryptography and Applications

Abstract

This report provides a concise introduction to Zero-Knowledge Proofs (ZKPs), a cryptographic technique that allows a prover to convince a verifier that a statement is true without revealing the underlying secret. The report first explains the motivation for privacy-preserving verification, then presents the formal relation-based model, the three core security properties, Sigma protocols, the Schnorr identification protocol, and the Fiat–Shamir transformation. These foundations support the later discussion of modern ZKP systems and applications such as blockchain scalability, digital identity, and confidential authentication. The goal is to connect the mathematical definitions from standard cryptographic literature with an intuitive explanation suitable for readers beginning to study ZKPs.

1 Introduction

Can someone prove that they know a secret without revealing the secret itself ?

This question captures the central idea of Zero-Knowledge Proofs (ZKPs). In conventional authentication, a user often proves legitimacy by revealing a password, a credential, or other sensitive information to a verifier. Although this approach is simple, it creates risks when the verifier is compromised, dishonest, or forced to store sensitive data.

ZKPs offer a different model: the prover demonstrates that a public statement is true while keeping the private witness hidden. In this sense, a ZKP separates *verification* from *disclosure*. The verifier gains confidence in the validity of the claim, but learns nothing beyond that validity. This idea is formalized through the notions of completeness, soundness, and zero-knowledge, which are introduced in the next section.

The report is organized as follows. Section 2 introduces the core mathematical model and uses the Schnorr identification protocol as a compact example of a Sigma protocol. Section 3 summarizes several modern ZKP systems, including zk-SNARKs, zk-STARKs, Bulletproofs, and PLONK. Section 4 discusses applications in blockchains, identity systems, and privacy-preserving authentication. The final section concludes with the main lessons and practical significance of ZKPs.

2 Fundamentals

Zero-Knowledge Proofs (ZKPs) allow a prover to convince a verifier that a statement is true without revealing the secret information that makes the statement true. Following the relation-based presentation used in standard cryptography texts such as Boneh and Shoup [1], this section introduces the minimum theoretical foundation needed before discussing modern ZKP systems.

2.1 Motivation

Traditional verification often requires disclosure. For example, a password-based login reveals the password to the server for checking, and a credential check may reveal more personal information than necessary. This creates two main risks:

- **Secret exposure:** the verifier or database may learn and store sensitive information.
- **Replay or misuse:** an attacker who observes or steals the secret may reuse it later.

A ZKP changes the goal of verification. Instead of sending the secret itself, the prover sends evidence that they possess a valid witness. This is useful in identity systems, privacy-preserving authentication, confidential transactions, and blockchain scaling systems where many computations can be verified through a compact proof.

2.2 Interactive Proof Systems

An interactive proof system is a protocol between two parties:

- **Prover ($\mathcal{P}$):** holds private information, called a witness, and tries to convince the verifier that a public statement is true.
- **Verifier ($\mathcal{V}$):** checks the proof using efficient computation and randomness, then outputs accept or reject.

The interaction is usually modeled as a sequence of messages. The verifier's random challenge is important because it prevents the prover from preparing a fixed response in advance. If the prover truly knows the witness, they should answer correctly; otherwise, they should fail except with small probability.

2.3 Relation-Based Mathematical Model

A ZKP is commonly described using a relation $R \subseteq X \times Y$:

- $y \in Y$ is the public **statement**.
- $x \in X$ is the private **witness**.
- $(x, y) \in R$ means that x is a valid witness for y .

The corresponding language is the set of statements that have at least one valid witness:

$$L_R = \{y \in Y \mid \exists x \in X \text{ such that } (x, y) \in R\}. \quad (1)$$

Thus, the language formalizes exactly the public claims for which some private witness exists; the proof should establish membership in L_R without identifying the witness.

A simple example is the discrete logarithm relation. Let G be a cyclic group of prime order q with generator g . The public statement is $u \in G$, and the witness is $\alpha \in \mathbb{Z}_q$ such that $u = g^\alpha$. The relation is

$$R_{DL} = \{(\alpha, u) \in \mathbb{Z}_q \times G \mid g^\alpha = u\}. \quad (2)$$

This relation captures the asymmetry used in many ZKPs: computing u from α is efficient, while recovering α from u is assumed hard in suitable groups. Therefore, a prover can demonstrate possession of α without disclosing it.

2.4 Core Security Properties

A zero-knowledge proof protocol must satisfy three essential properties:

- **Completeness:** if the statement is true and both parties follow the protocol, the verifier accepts. Ideally,

$$\Pr[\langle \mathcal{P}(x, y), \mathcal{V}(y) \rangle = \text{accept}] = 1. \quad (3)$$

- **Soundness:** if the statement is false, a cheating prover should not convince the verifier except with negligible probability. In proof-of-knowledge settings, soundness is often strengthened by requiring an extractor that can recover a witness from a successful cheating prover.
- **Zero-knowledge:** the verifier learns nothing beyond the truth of the statement. Formally, there exists a simulator that can generate a transcript indistinguishable from a real protocol execution without knowing the witness.

These properties explain the central balance of ZKPs: honest provers can convince verifiers, dishonest provers cannot easily cheat, and private witnesses remain hidden. Together, these properties separate correctness, resistance to cheating, and privacy of the witness.

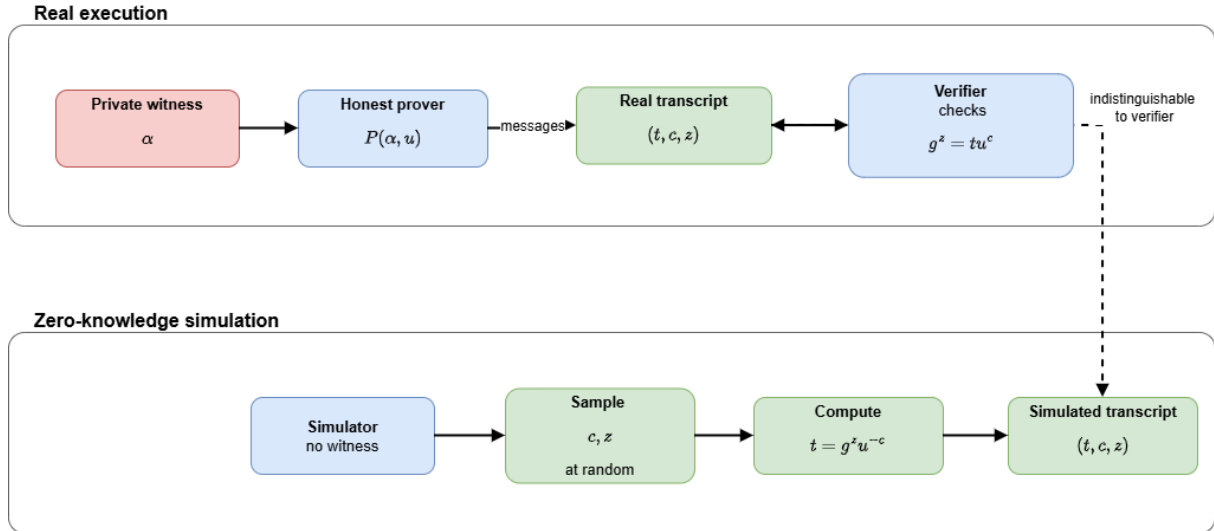


Figure 1: Simulation intuition for zero-knowledge: a simulator can generate an accepting transcript with the same observable distribution as a real execution, even without the witness.

2.5 Sigma Protocols

A Sigma protocol is a three-message interactive proof that is widely used as a building block for modern ZKP constructions. Its structure is:

1. **Commitment:** $\mathcal{P}$ sends a commitment t .
2. **Challenge:** $\mathcal{V}$ samples a random challenge c .
3. **Response:** $\mathcal{P}$ replies with z , and $\mathcal{V}$ checks a public verification equation.

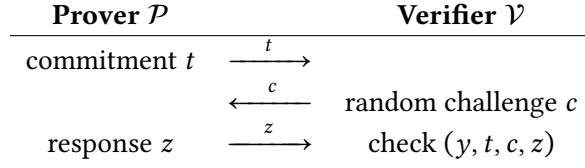


Figure 2: Standard three-move structure of a Sigma protocol.

Sigma protocols are attractive because they are simple, efficient, and often satisfy completeness, special soundness, and honest-verifier zero-knowledge (HVZK). Because the commitment precedes the challenge, the prover cannot tailor the first message to a known verifier query; this ordering is central to soundness.

2.6 Schnorr Identification Protocol

The Schnorr identification protocol is a canonical Sigma protocol based on the discrete logarithm problem [2]. Let $u = g^\alpha$ be the public key and let α be the prover's secret witness. Here, r randomizes the response, so z verifies algebraically while not revealing α by itself.

1. **Commitment:** the prover samples $r \leftarrow_R \mathbb{Z}_q$, computes

$$t = g^r, \tag{4}$$

and sends t to the verifier.

2. **Challenge:** the verifier samples $c \leftarrow_R \mathbb{Z}_q$ and sends c to the prover.
3. **Response:** the prover computes

$$z = r + c\alpha \pmod{q} \tag{5}$$

and sends z to the verifier.

4. **Verification:** the verifier accepts if

$$g^z = t \cdot u^c. \tag{6}$$

Completeness follows directly because $g^z = g^{r+c\alpha} = g^r(g^\alpha)^c = tu^c$. Special soundness comes from the fact that two accepting transcripts with the same t but different challenges $c \neq c'$ allow extraction of the secret:

$$\alpha = (z - z') \cdot (c - c')^{-1} \pmod{q}. \tag{7}$$

HVZK is shown by simulation: choose random $c, z \leftarrow_R \mathbb{Z}_q$ and set $t = g^z u^{-c}$. The resulting transcript (t, c, z) verifies correctly and has the same distribution as an honest execution. The

simulation shows that accepting transcripts do not inherently contain witness-dependent information.

2.7 Fiat-Shamir Transformation

Interactive protocols require the prover and verifier to exchange messages online. The Fiat-Shamir transformation removes this interaction by replacing the verifier's random challenge with a hash value [3]. For the Schnorr protocol, the prover computes

$$c = H(m, u, t), \quad (8)$$

where m is the context or message being proven. The proof can then be represented compactly as (t, z) or (c, z) .

To clearly clarify this transformation, Figure 3 illustrates the structural shift from an online interactive protocol to a single-message, non-interactive argument.

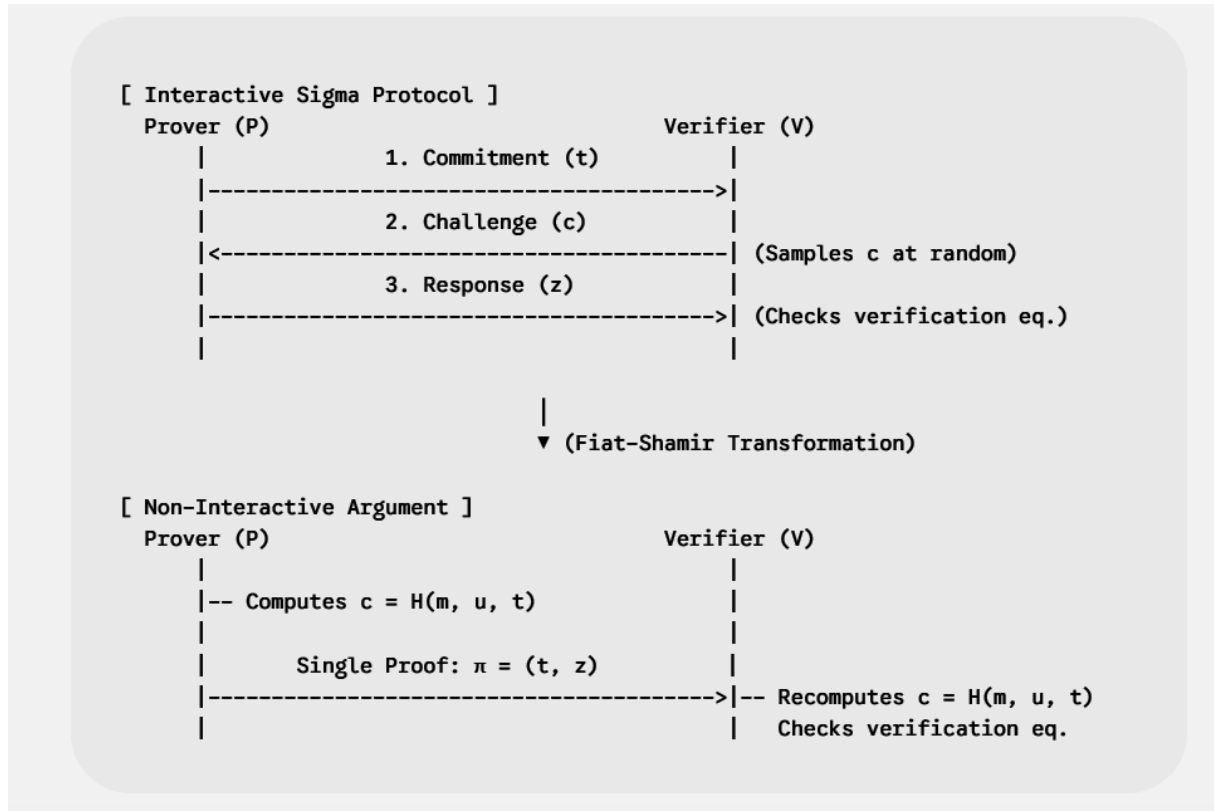


Figure 3: Comparison between the interactive three-move exchange and the non-interactive Fiat-Shamir workflow.

During verification, the verifier recomputes the challenge from the public data and checks the same Schnorr equation. This transformation is a key bridge from interactive proofs to non-interactive proof systems and digital signatures. It also motivates why hash functions and the random-oracle model appear frequently in practical ZKP systems. The hash therefore acts as a public source of challenge unpredictability, binding the proof to (m, u, t) .

3 Modern Zero-Knowledge Proof Systems

While foundational protocols like Schnorr and the Fiat-Shamir transformation laid the theoretical groundwork for ZKP, they are primarily limited to proving specific algebraic relations (e.g., knowledge of a discrete logarithm). Modern cryptography requires proving arbitrary computational statements. This section explores the evolution of general-purpose ZKP systems.

3.1 Motivation for Modern ZKP

The transition from classical ZKPs to modern systems is driven by three main requirements:

1. **General-Purpose Computation:** The ability to prove the correct execution of any arbitrary program or algorithm, typically modeled as Boolean or Arithmetic circuits (Circuit Satisfiability), rather than just simple cryptographic puzzles.
2. **Succinctness:** The proof size must be extremely small (often just a few hundred bytes), and the verification time must be on the order of milliseconds, regardless of how complex the underlying computation is.
3. **Non-Interactivity:** As discussed in Section 2, modern applications like blockchains require non-interactive proofs that can be verified asynchronously by anyone.

These requirements led to the development of powerful cryptographic primitives such as SNARKs, STARKs, Bulletproofs, and PLONK.

3.2 From Programs to Algebraic Constraints

The key idea behind modern ZKP systems is to convert a program into a finite set of algebraic constraints. Let y denote the public input and x denote the private witness. Instead of revealing x , the prover shows the existence of a witness such that

$$C(y, x) = 1, \tag{9}$$

where C is a circuit representation of the computation. In an arithmetic-circuit view, every wire stores a field element and every gate enforces additions or multiplications over a finite field $\mathbb{F}_p$.

A common representation is a **Rank-1 Constraint System (R1CS)**. For a witness vector

$$\mathbf{w} = (1, y, x, w_1, w_2, \dots, w_k), \tag{10}$$

each constraint has the form

$$\langle \mathbf{a}_i, \mathbf{w} \rangle \cdot \langle \mathbf{b}_i, \mathbf{w} \rangle = \langle \mathbf{c}_i, \mathbf{w} \rangle, \quad i = 1, \dots, m. \tag{11}$$

Thus, proving correct execution becomes proving that all m equations in (11) are satisfied.

Example. Suppose the public statement is “ $y = x^3 + x + 5$ for some private x ”. The prover does not reveal x ; it only proves that such an x exists. Introduce intermediate wires

$$z_1 = x^2, \quad z_2 = z_1 x, \quad z_3 = z_2 + x, \quad y = z_3 + 5. \quad (12)$$

The witness is $\mathbf{w} = (1, y, x, z_1, z_2, z_3)$. For example, if $x = 3$, then

$$z_1 = 9, \quad z_2 = 27, \quad z_3 = 30, \quad y = 35. \quad (13)$$

The verifier learns only that the arithmetic relation is satisfied; it does not learn the hidden value x .

3.3 zk-SNARK

zk-SNARK stands for *Zero-Knowledge Succinct Non-Interactive Argument of Knowledge*. It is arguably the most widely deployed ZKP technology today (e.g., used in Zcash).

- **Underlying Mathematics:** SNARKs typically rely on pairing-based cryptography (Bilinear Maps over Elliptic Curves) and represent computations using Rank-1 Constraint Systems (R1CS) and Quadratic Arithmetic Programs (QAP).
- **Key Characteristics:** They offer constant proof sizes ($O(1)$, typically around 288 bytes) and constant verification time, meaning verifying a multi-million-gate circuit takes the same time as verifying a trivial one.
- **The Trusted Setup Problem:** The major drawback of early SNARKs (like Groth16) is the requirement for a per-circuit *Trusted Setup*. A group of participants must generate a Common Reference String (CRS) using random secrets (often called “toxic waste”). If these secrets are not destroyed, a malicious party could forge false proofs.

In a QAP-based SNARK, the R1CS constraints are encoded as polynomials. If $A(X)$, $B(X)$, and $C(X)$ are interpolating polynomials derived from the left, right, and output terms of the constraints, then the prover demonstrates that

$$A(X)B(X) - C(X) = H(X)Z(X), \quad (14)$$

where $Z(X)$ is the vanishing polynomial over all constraint points. Equation (14) means that every original circuit constraint is satisfied. Pairing-based commitments then allow the verifier to check this polynomial identity at a hidden evaluation point without reading the entire computation.

For Groth16-style proofs, the final proof can be viewed abstractly as only three group elements,

$$\pi = (A_\pi, B_\pi, C_\pi), \quad (15)$$

and verification reduces to a small number of bilinear-pairing equations. This is why SNARK proofs are compact even when the circuit contains millions of gates.

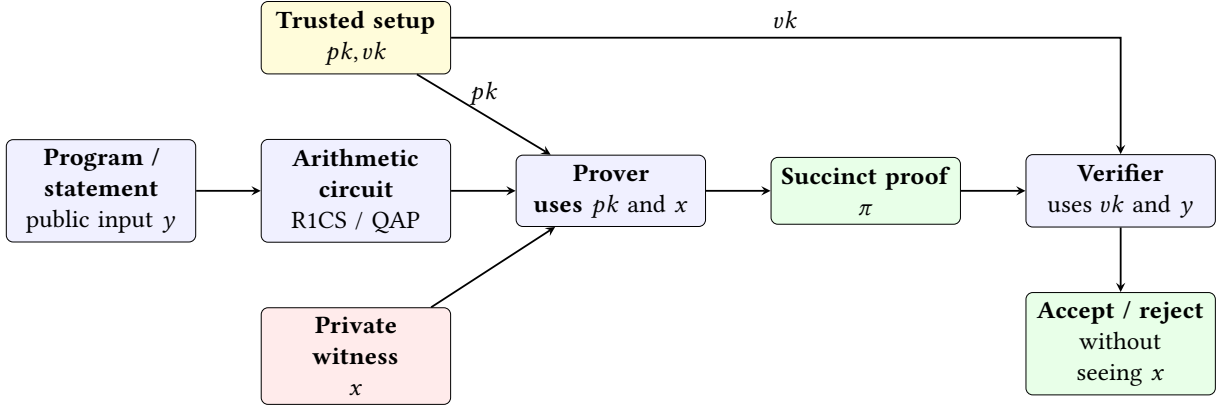


Figure 4: High-level zk-SNARK proving workflow: the expensive computation is encoded into constraints and proven once, while verification only checks a compact proof against public data.

3.4 zk-STARK

zk-STARK stands for *Zero-Knowledge Scalable Transparent Arguments of Knowledge*, pioneered by Eli Ben-Sasson and the StarkWare team.

- **Transparency:** Unlike SNARKs, STARKs do not require any trusted setup. They rely solely on publicly verifiable randomness, making them "transparent".
- **Post-Quantum Security:** STARKs do not rely on Number Theoretic assumptions (like Elliptic Curves or Discrete Logarithms) which are vulnerable to Shor's algorithm. Instead, they rely only on collision-resistant hash functions, making them plausibly post-quantum secure.
- **Scalability via FRI:** STARKs use a protocol called FRI (Fast Reed-Solomon Interactive Oracle Proofs of Proximity). While prover and verifier times scale quasi-linearly and poly-logarithmically respectively, the proof sizes are significantly larger than SNARKs (typically 40 to 100 KB).

STARKs describe computation using an **Algebraic Intermediate Representation (AIR)**. Instead of first building an R1CS, the prover records an execution trace and proves that adjacent rows satisfy transition constraints. For a toy computation

$$s_{i+1} = s_i^2 + c, \quad (16)$$

the trace is $(s_0, s_1, \dots, s_T)$ and correctness is expressed as

$$s_{i+1} - s_i^2 - c = 0, \quad i = 0, \dots, T - 1. \quad (17)$$

The trace values are interpolated into a low-degree polynomial $f(X)$ over a domain D . If g moves from one row of the trace to the next, then the transition constraint becomes the polynomial condition

$$f(gX) - f(X)^2 - c = 0 \quad \text{for all } X \in D. \quad (18)$$

FRI is then used to prove that the committed polynomials are close to low-degree polynomials, while Merkle trees allow the verifier to check only a few randomly sampled positions.

3.5 Bulletproofs

Introduced by Bünz et al. in 2017 [4], **Bulletproofs** are short non-interactive zero-knowledge proofs designed specifically to prove that a committed value lies within a certain range (Range Proofs), without requiring a trusted setup.

- **Inner-Product Argument:** The core mathematical innovation is a highly efficient inner-product argument that shrinks the proof size logarithmically ($O(\log N)$) relative to the size of the computation.
- **Application:** Because of their extremely small proof size for simple operations and lack of a trusted setup, Bulletproofs are heavily utilized in the Monero cryptocurrency to conceal transaction amounts (Confidential Transactions).
- **Trade-off:** The verification time scales linearly ($O(N)$) with the circuit size, making them less suitable for highly complex smart contract computations compared to SNARKs.

The typical use case is a range proof over a Pedersen commitment

$$C = vG + rH, \quad (19)$$

where v is the hidden value and r is random blinding. To prove $0 \leq v < 2^n$, the prover represents v in binary,

$$v = \sum_{i=0}^{n-1} 2^i b_i, \quad b_i \in \{0, 1\}, \quad (20)$$

and must convince the verifier that every bit is valid:

$$b_i(b_i - 1) = 0, \quad i = 0, \dots, n - 1. \quad (21)$$

Bulletproofs compress these bit constraints into an inner-product argument, so the proof grows logarithmically rather than linearly with n .

3.6 PLONK

PLONK (Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge) was introduced by Gabizon, Williamson, and Bowman to address the strict trusted setup of Groth16.

- **Universal and Updatable Setup:** PLONK requires a trusted setup, but it is *universal*. A single setup generation (like the KZG ceremony) can be used for any program of up to a certain size. Furthermore, it is *updatable*, meaning anyone can continually add randomness to the setup to increase its security.
- **Custom Gates:** PLONK uses polynomial commitments (typically KZG) and models

circuits using copy constraints (permutation arguments). It allows for "custom gates," enabling optimized performance for specific cryptographic operations (like hashing within the circuit).

PLONK expresses each row of the circuit with selector polynomials. A standard gate has the form

$$q_L a + q_R b + q_O c + q_M ab + q_C = 0, \quad (22)$$

where a , b , and c are wire values and q_L , q_R , q_O , q_M , q_C are selectors. For example, choosing

$$q_M = 1, \quad q_O = -1, \quad q_L = q_R = q_C = 0 \quad (23)$$

makes (22) become $ab - c = 0$, which is a multiplication gate. Choosing $q_L = q_R = 1$, $q_O = -1$, and $q_M = q_C = 0$ gives $a + b - c = 0$, an addition gate. Copy constraints, such as reusing the same wire value across different rows, are enforced using a permutation argument rather than writing separate equality constraints one by one.

3.7 Comparison of Modern ZKP Systems

To summarize, Table 1 provides a comparative overview of the leading modern ZKP systems.

Protocol	Math Assumption	Trusted Setup	Proof Size	Prover Time	Verifier Time
Groth16 (SNARK)	Elliptic Curve Pairings	Yes (Per Circuit)	≈ 288 bytes ($O(1)$)	$O(N \log N)$	≈ 2 ms ($O(1)$)
PLONK	Pairings + KZG	Universal/Updatable	≈ 1 KB ($O(1)$)	$O(N \log N)$	≈ 3 ms ($O(1)$)
zk-STARK	Hash Functions	No (Transparent)	40 – 100 KB ($O(\log^2 N)$)	$O(N \log N)$	$O(\log^2 N)$
Bulletproofs	Discrete Logarithm	No (Transparent)	≈ 1.5 KB ($O(\log N)$)	$O(N \log N)$	$O(N)$

Table 1: Comparison of Modern Zero-Knowledge Proof Systems. N represents the complexity (number of gates) of the computational circuit.

4 Applications of Zero-Knowledge Proofs

The transition of ZKP from theoretical mathematics to practical software has unlocked unprecedented capabilities in cryptography, particularly in bridging the gap between absolute privacy and strict verifiability.

4.1 Blockchain and zk-Rollups

The most prominent application of modern ZKPs is in blockchain technology, addressing the "Blockchain Trilemma" (Scalability, Security, and Decentralization).

- **Privacy Coins:** Networks like Zcash use zk-SNARKs to completely encrypt sender, receiver, and transaction amounts while proving to the network that no double-spending occurred and no coins were created out of thin air.

- **zk-Rollups for Scalability:** Ethereum and other Layer-1 networks suffer from low throughput. A zk-Rollup (e.g., Starknet, zkSync) shifts computation off-chain. An aggregator processes tens of thousands of transactions, executes them, and generates a single succinct ZKP (a validity proof). The main blockchain only needs to verify this tiny proof, exponentially increasing throughput and lowering gas fees without compromising Layer-1 security.

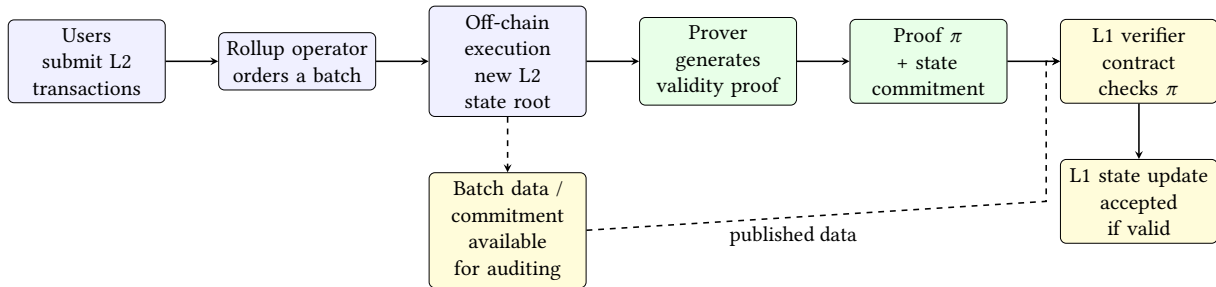


Figure 5: Simulation of a zk-rollup validity proof flow: transactions are executed off-chain, while Layer 1 only verifies a compact proof and the resulting state commitment.

4.2 Privacy-Preserving Authentication

Traditional authentication relies on servers storing hashed passwords. If the server is breached, user credentials are at risk. ZKP enables "Passwordless Authentication" with absolute security. A user's device can generate a proof that it knows the secret key associated with an account. The server verifies the proof without ever receiving or storing the secret itself. This architecture completely eliminates the risk of centralized server data breaches (honeypots).

4.3 Digital Identity

In the paradigm of Self-Sovereign Identity (SSI), users control their own data. ZKPs enable **Selective Disclosure** and **Predicate Proofs**. For instance, to access an age-restricted service, a user typically provides a passport, revealing their exact birthdate, name, and address. Using a ZKP, the user can cryptographically prove the mathematical statement: $\text{Current_Year} - \text{Birth_Year} \geq 18$, based on a government-signed credential. The verifier learns that the user is over 18, but learns absolutely nothing else about their identity.

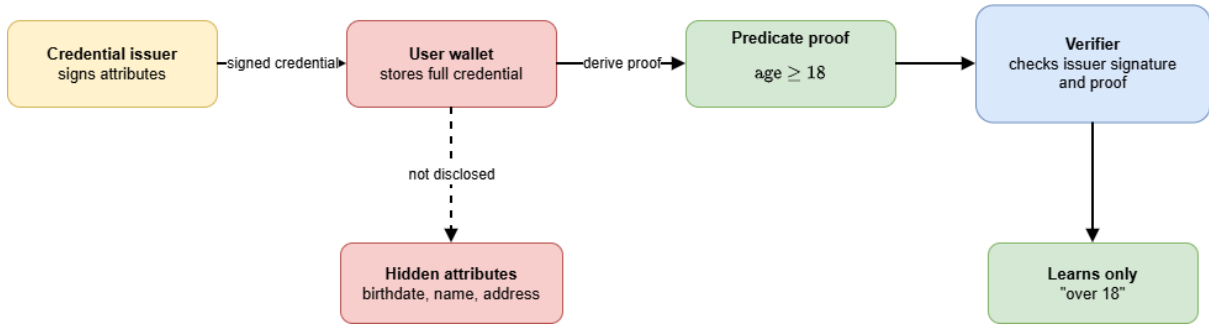


Figure 6: Selective disclosure with ZKP: a verifier can check a predicate over a signed credential without receiving the underlying personal attributes.

4.4 Anonymous Voting

Secure electronic voting requires two seemingly contradictory properties: *Verifiability* (every vote is counted correctly) and *Privacy/Coercion Resistance* (no one can see who voted for whom). ZKP systems elegantly solve this. A voter encrypts their ballot and attaches a ZKP proving that the encrypted ballot contains a valid vote (e.g., exactly one candidate is selected, and the voter is registered). The system tallies the encrypted votes using homomorphic encryption and proves the final tally is correct, all while maintaining absolute voter anonymity.

5 Conclusion

5.1 Limitations

Despite their revolutionary potential, ZKP systems face several ongoing challenges that remain active areas of research:

- **Prover Computational Overhead:** Generating a proof (Prover time) involves complex mathematical operations (like multi-scalar multiplications and Fast Fourier Transforms). For massive circuits, this requires significant memory and high-performance computing resources, creating a bottleneck for resource-constrained devices like smartphones.
- **Trusted Setup Risks:** Systems relying on a per-circuit trusted setup (like Groth16) carry the systemic risk of cryptographic compromise if the setup ceremony is breached.
- **Quantum Vulnerability:** SNARKs, PLONK, and Bulletproofs rely on elliptic curves and discrete logarithms, which are theoretically vulnerable to Shor’s algorithm on a future sufficiently powerful quantum computer.

5.2 Summary

Zero-Knowledge Proofs have evolved from an esoteric concept in the 1980s into a foundational pillar of modern applied cryptography. By defining rigorous mathematical models based on interactive proof systems and establishing core security properties—Completeness, Soundness,

and Zero-Knowledge—researchers have created secure frameworks like Sigma protocols and the Fiat-Shamir transformation.

The advent of succinct, non-interactive systems such as SNARKs, STARKs, and PLONK has bridged the gap between theory and practice. Today, ZKPs provide robust solutions for blockchain scalability, privacy-preserving authentication, and sovereign digital identity. While computational overhead and post-quantum transitions present ongoing challenges, ZKP technology fundamentally redefines the boundaries of digital trust, proving that verification no longer requires the sacrifice of privacy.

References

- [1] V. S. Dan Boneh, *A Graduate Course in Applied Cryptography*. 2023. [Online]. Available: <https://toc.cryptobook.us/book.pdf>
- [2] C.-P. Schnorr, “Efficient signature generation by smart cards,” *Journal of Cryptology*, vol. 4, no. 3, pp. 161–174, 1991.
- [3] A. Fiat and A. Shamir, “How to prove yourself: Practical solutions to identification and signature problems,” in *Advances in Cryptology – CRYPTO ’86*, 1986, pp. 186–194.
- [4] B. Bünz, J. Bootle, D. Boneh, A. Poelstra, P. Wuille, and G. Maxwell, *Bulletproofs: Short proofs for confidential transactions and more*, 2017. Cryptology ePrint Archive: [2017/1066](https://eprint.iacr.org/2017/1066). [Online]. Available: <https://eprint.iacr.org/2017/1066>